



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

Umetna inteligenca v avtonomni vožnji

Prva različica aplikacije za 3D interaktivno vizualizacijo

Projektno delo - Računalniška grafika

Maribor, december 2025

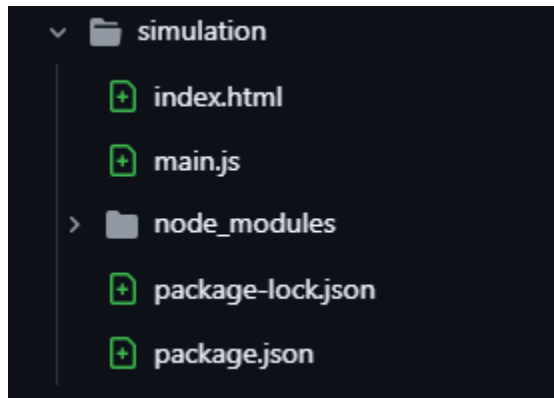
Miloš Avakumović,

Vedran Dojčinović

Slađana Petrović

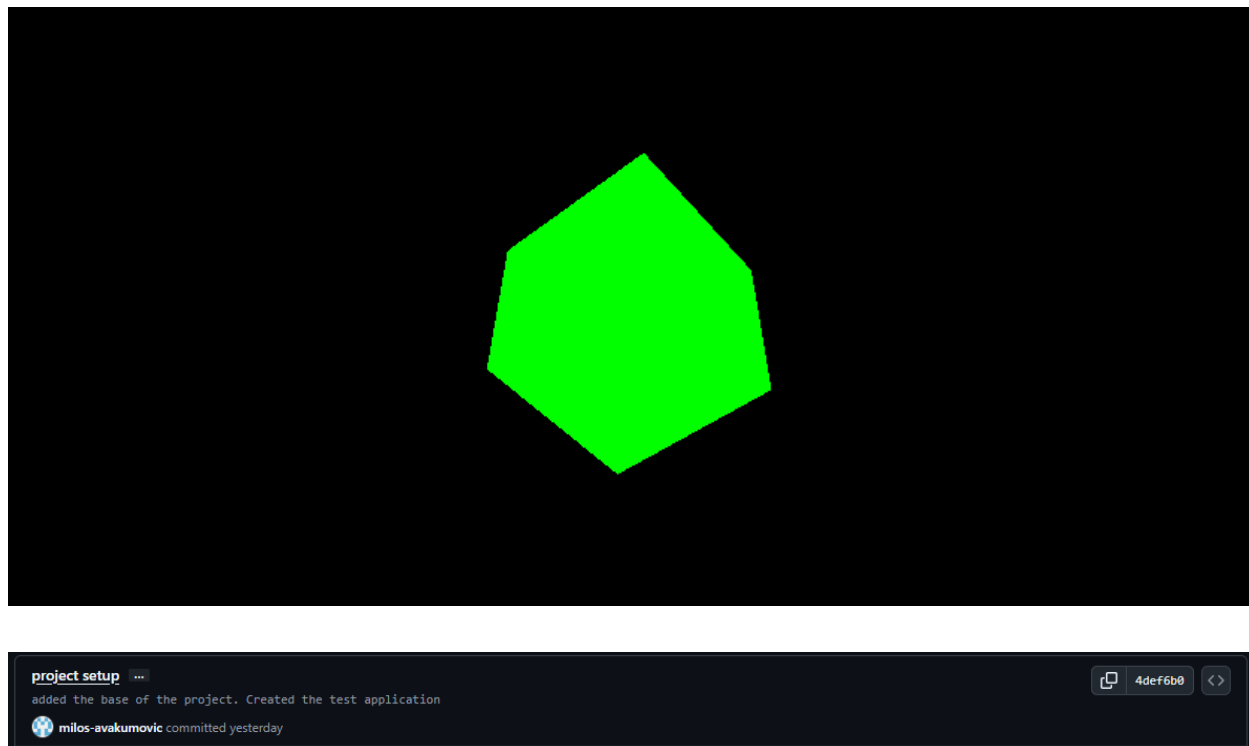
Vzpostavitev strukture projekta in vključitev knjižnic

Pripravil sem osnovno strukturo projekta, vključil module knjižnice Three.js ter vzpostavil lokalni razvojni strežnik z uporabo orodja Vite. Osnovna struktura projekta je bila naslednja:



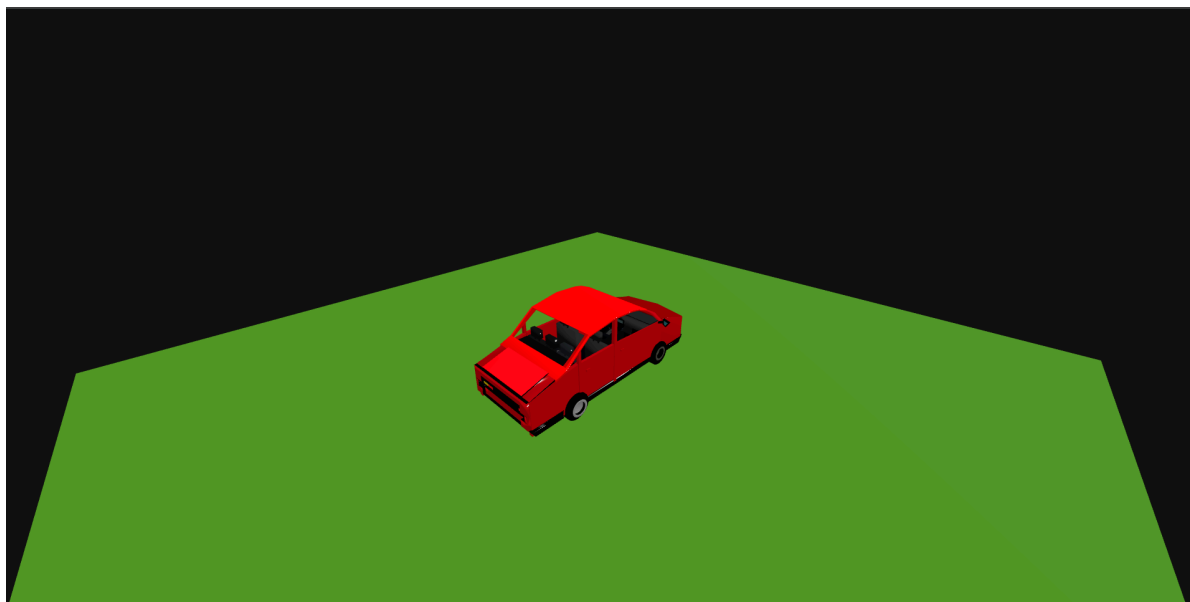
Priprava osnovne 3D scene

Po vzpostavitvi strukture programa ter namestitvi vseh potrebnih knjižnic in orodij sem ustvaril preprosto testno sceno. Kodo za testno sceno sem povzel iz uradne dokumentacije knjižnice Three.js.



Vključitev 3D modelov in oblikovanje 3D scene

Ustvaril in preizkusil sem testno aplikacijo, pri čemer sem sledil uradni dokumentaciji knjižnice Three.js. Nato sem začel dodajati 3D-modele v sceno. Prvi dodani model je bil avtomobil, katerega avtorica je kolegica Nejla Perenda. Zatem sem v notranjost avtomobila dodal tudi krmilno enoto, ki sem jo sam modeliral.



adding car + control unit ...

Added a model of a car ("rac_grafika_model_armatura2.obj") created by Nejla Perenda. Added my control unit box ("model_1.obj") into the scene. Played with camera settings.



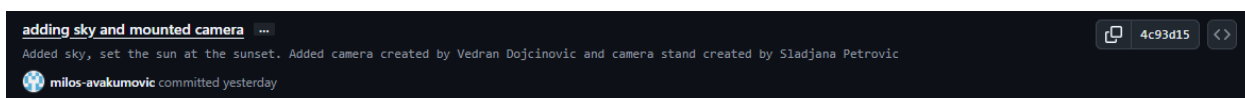
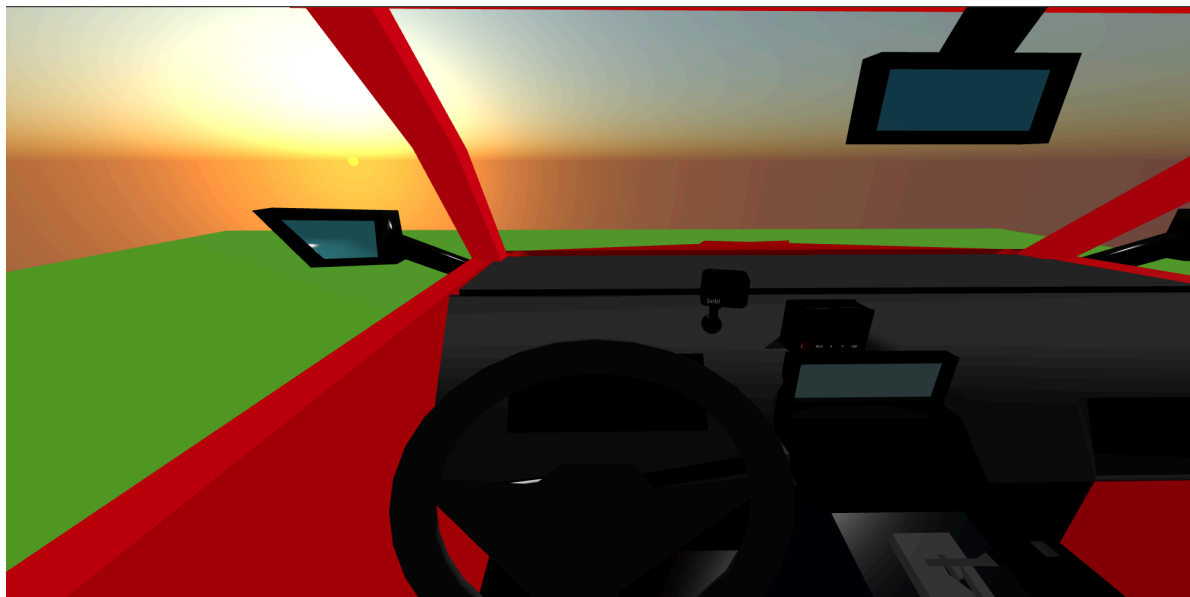
milos-avakumovic committed yesterday



c344f8c

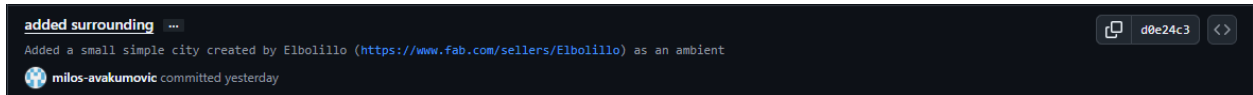


Nato sem dodal kamero, usmerjeno proti vozniku, ki jo je ustvaril Vedran Dojčinović, ter nosilec kamere, ki ga je modelirala Slađana Petrović. Dodal sem tudi videz neba z uporabo vnaprej pripravljenih objektov knjižnice Three.js, pri čemer sem sonce nastavil nizko nad obzorje, tako da simulira sončni zahod.

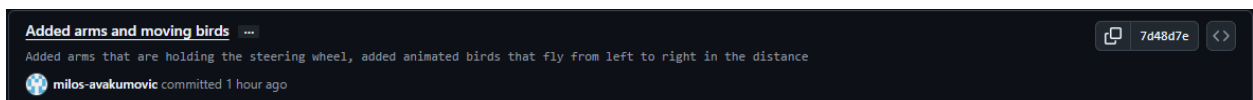


Naslednji korak je bilo dodajanje okolice. Model manjšega naselja sem našel na spletu, avtor modela pa je Elbolillo (<https://www.fab.com/sellers/Elbolillo>). Avtomobil sem postavil na eno izmed ulic v naselju.





Na koncu sem dodal še roke, ki držijo volan, ter ptice, ki letijo v daljavi. Oba modela sta bila prenesena s spleta; avtor modela rok je DJMaesen, avtor modela jate ptic pa moizmuhammad373. Model ptic že vsebuje eno animacijo (mahanje s krili), poleg tega pa sem dodal še translacijsko gibanje jate z desne proti levi strani scene.



Interakcija z 3D okoljem in vizualni prikaz stanja voznika

Namen vizualne kontrolne enote je prikaz trenutnega stanja voznika, ki ga zazna sistem umetne inteligence za zaznavanje utrujenosti. Kontrolna enota je nameščena znotraj vozila in omogoča hiter ter intuitiven vpogled v stanje voznika med vožnjo.

Vizualni prikaz je realiziran s pomočjo barvnega LED indikatorja in zaslona na kontrolni enoti, ki se dinamično prilagajata glede na zaznano stanje.

Model kontrolne enote

Kontrolna enota je bila pripravljena v programskem okolju Blender in vključena v 3D sceno aplikacije s pomočjo knjižnice Three.js. Model je sestavljen iz več ločenih komponent, ki omogočajo interakcijo in dinamično spreminjanje videza: zaslon (*Ekran*), LED indikator (*LED trak*), funkcijski gumbi.

Ločena struktura modela omogoča neposredno manipulacijo posameznih delov v 3D sceni, kar je ključno za vizualni prikaz stanja sistema.

Implementacija interakcije v Three.js

Interakcija s kontrolno enoto je implementirana v okolju Three.js z uporabo lastnega modula `driverIndicators.js`. Modul omogoča iskanje ustreznih objektov v 3D sceni ter dinamično spreminjanje njihovih materialov glede na vhodni signal.

Objekti so v sceni identificirani z uporabo njihovih imen, pridobljenih iz izvoznega modela. Po prejemu signala se spremeni barva oziroma emisijska komponenta materiala LED indikatorja in zaslona, s čimer se doseže jasen vizualni odziv sistema.

Implementacija je zasnovana modularno, kar omogoča enostavno nadgradnjo in integracijo z ostalimi deli sistema.

Sistem uporablja enostavno preslikavo vhodnih signalov v vizualne prikaze:

00 - Buden - Zelena barva

01 - Utrujen - Rumena barva

10 - Zaspan - Rdeča barva

```
function applyDriverState(code) {
  console.log("applyDriverState:", code);

  if (code === "00") { // budan
    setEmissiveOrColor(lamp, 0x00ff00, 2.0);
    setEmissiveOrColor(screen, 0x003300, 2.5);
  }
  if (code === "01") { // utrujen
    setEmissiveOrColor(lamp, 0xffff00, 2.0);
    setEmissiveOrColor(screen, 0x333300, 2.5);
  }
  if (code === "10") { // zaspan
    setEmissiveOrColor(lamp, 0xff0000, 2.0);
    setEmissiveOrColor(screen, 0x330000, 2.5);
  }
}
```

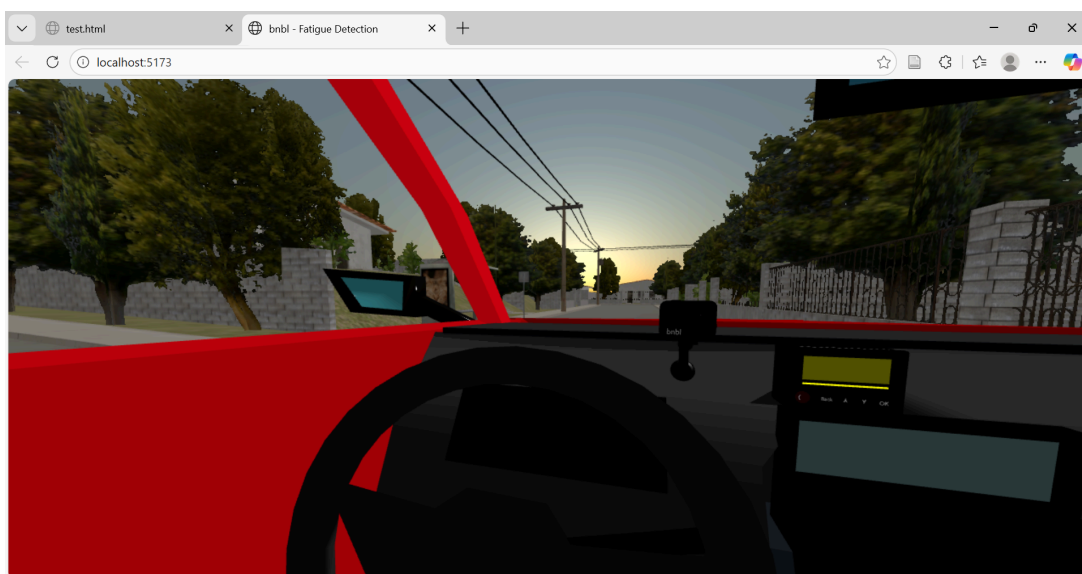
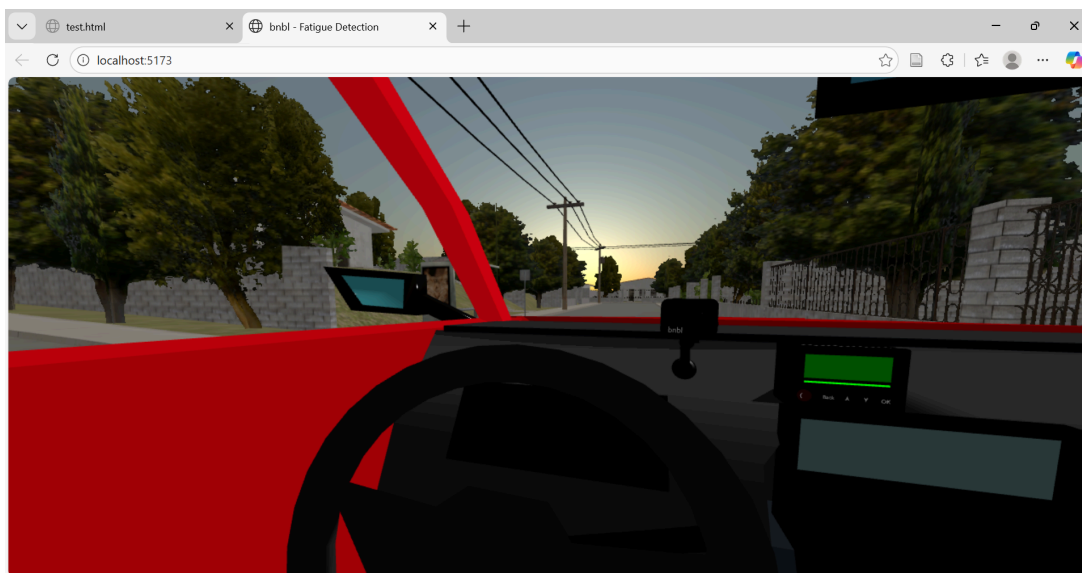
Testiranje

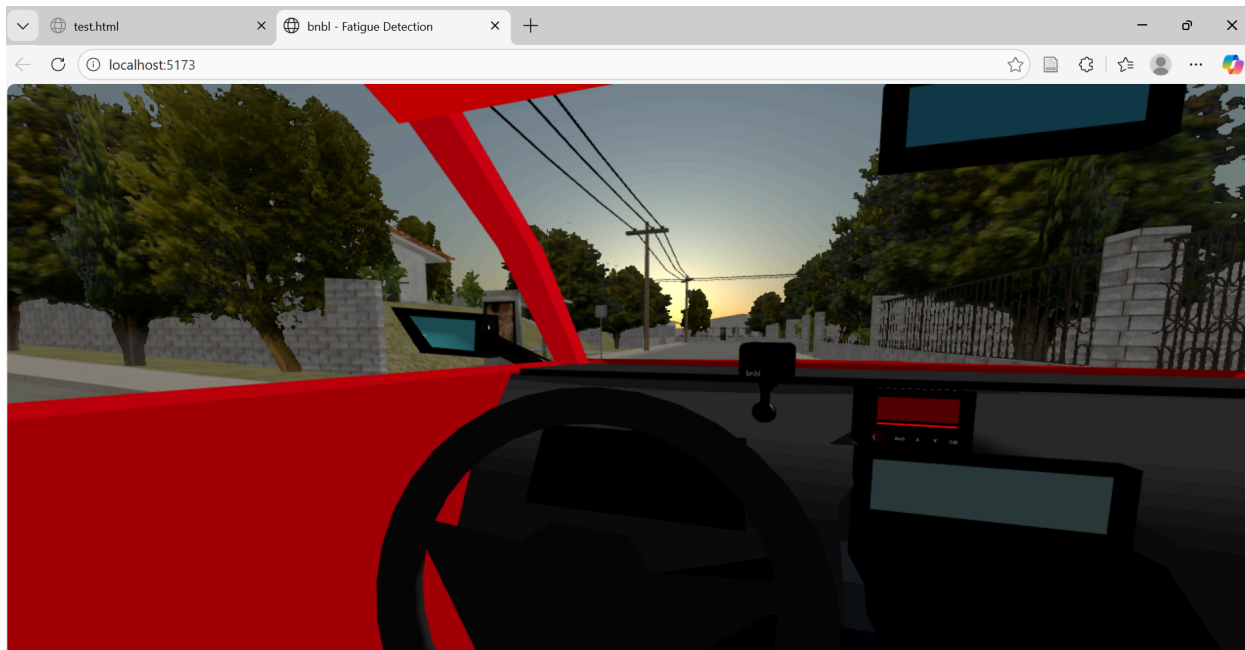
Funkcionalnost je bila testirana z uporabo simuliranih vhodnih signalov, sproženih preko tipkovnice (tipke 0, 1 in 2). S tem je bilo omogočeno testiranje vizualnega odziva sistema preden je bilo vključeno v komunikacijo in pošiljanje strežniku ali Docker okolju. S tem sem na začetku zagotovila da funkcionalnost deluje, torej je pravilno implementirana, preden sem po zagonu WebSocket strežnika isti mehanizem uporabila za prikaz realnih podatkov. Testiranje je potrdilo pravilno delovanje modula, saj se LED indikator in zaslon ob vsakem vhodnem signalu ustrezno posodobita.

Ustvarila sem skripto stateInputDemo.js za preverjanje prvobitnega delovanja.

```
export function attachStateInputDemo(onState) {
  window.addEventListener("keydown", (e) => {
    console.log("KEY:", e.key);
    if (e.key === "0") onState("00");
    if (e.key === "1") onState("01");
    if (e.key === "2") onState("10");
  });
}
```

Spodaj so posnetki za posamezna stanja:





Integracija z ostalimi deli sistema

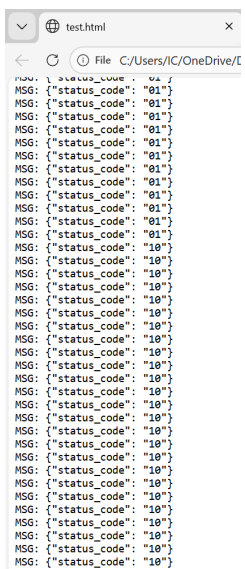
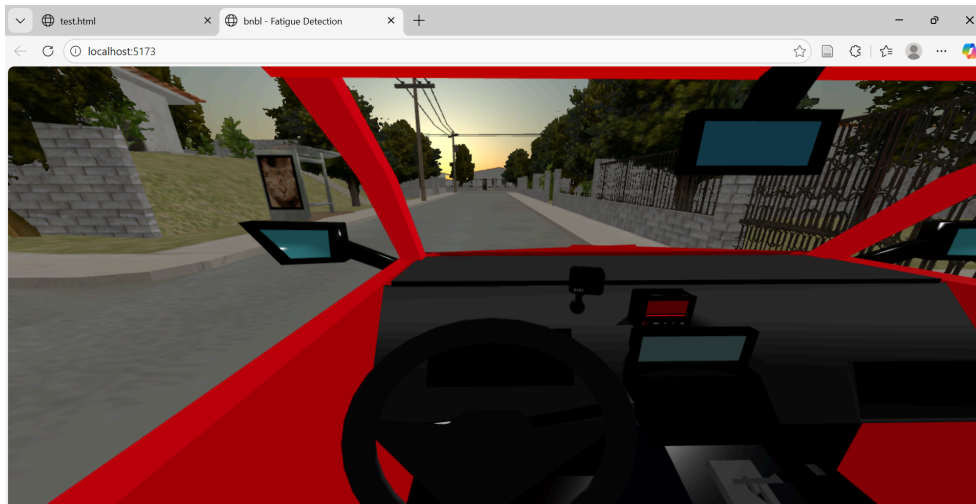
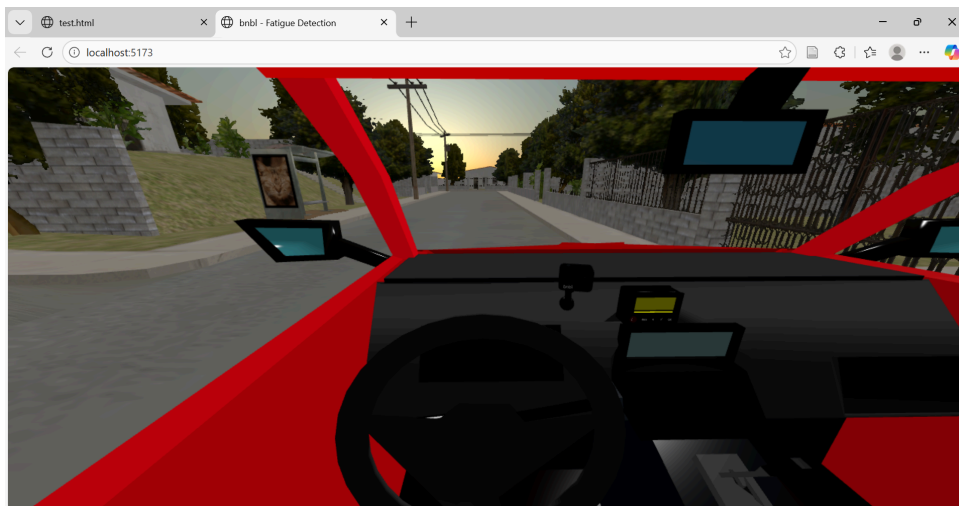
Implementirani modul je pripravljen za neposredno povezavo z zalednim sistemom za zaznavanje utrujenosti voznika. Po vzpostavitvi komunikacije preko WebSocket ali MQTT protokola bo vizualni prikaz v realnem času odražal dejanske rezultate sistema umetne inteligence. Zasnova omogoča enostavno zamenjavo simuliranih vhodnih signalov z realnimi podatki brez sprememb v logiki vizualizacije.

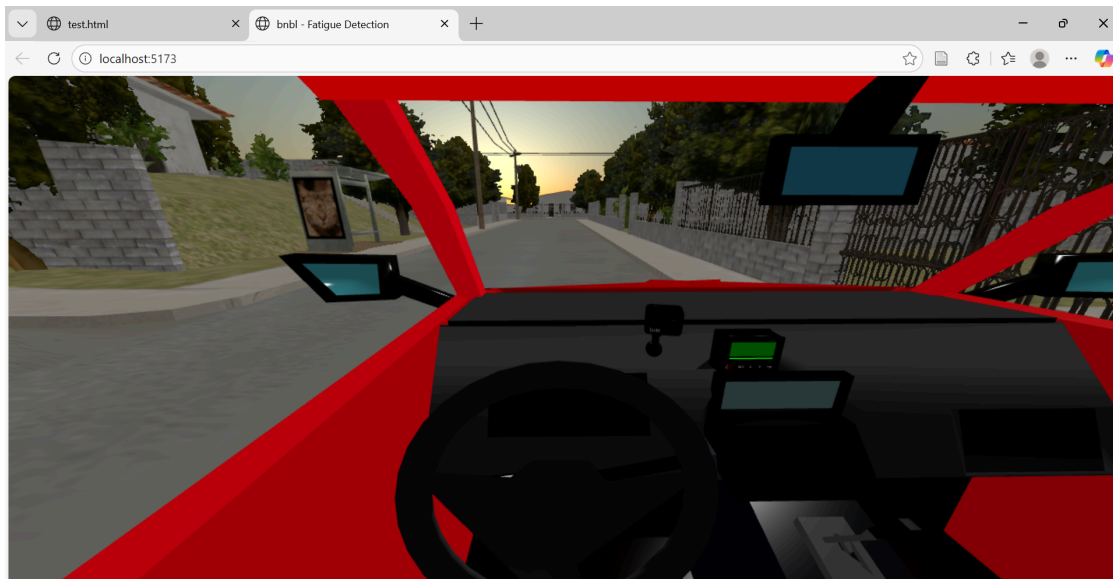
Po vzpostavitvi WebSocket povezave (Docker strežnik) aplikacija v realnem času prejema sporočila v obliki JSON (`{"status_code": "00/01/10"}`). Prejeto stanje se parsira in posreduje modulu `driverIndicators`, ki na kontrolni enoti spremeni svetlost/barvo LED indikatorja ter zaslona.

Barvni LED indikator in zaslon na kontrolni enoti omogočata hiter vpogled v trenutno stanje voznika (buden, utrujen ali zaspan) brez dodatne kognitivne obremenitve. Takšen prikaz je še posebej pomemben v varnostno kritičnih sistemih, kjer mora biti informacija posredovana jasno, in v realnem času.

Poleg tega vizualizacija v 3D okolju omogoča učinkovito testiranje in demonstracijo delovanja celotnega sistema, saj povezuje zaznavanje, obdelavo podatkov in uporabniški vmesnik v enotno celoto. S tem projekt presega zgolj teoretično obdelavo podatkov in ponuja celovit prikaz delovanja sistema v kontekstu realne uporabe.

Prikaz po integraciji v realnočasovnom pošiljanju:





```
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
MSG: {"status_code": "00"}
```

Logika z katero sem zamenjala demo:

```
const indicators = createDriverIndicators(controlBoxObj, {
  screenName: "Ekran_Material.004",
  lampName: "LED_trak_LED_bar",
});

// WebSocket real-time data iz Dockera
const ws = new WebSocket("ws://172.25.86.22:3001");

ws.onopen = () => console.log("WS OPEN");
ws.onerror = (e) => console.log("WS ERROR", e);
ws.onclose = () => console.log("WS CLOSE");

// Prihaja JSON: {"status_code": "10"}
ws.onmessage = (e) => {
  try {
    const msg = JSON.parse(e.data);
    const code = String(msg.status_code).trim(); // "00"/"01"/"10"
    indicators.applyDriverState(code);
  } catch (err) {
    console.warn("Bad WS message:", e.data, err);
  }
};

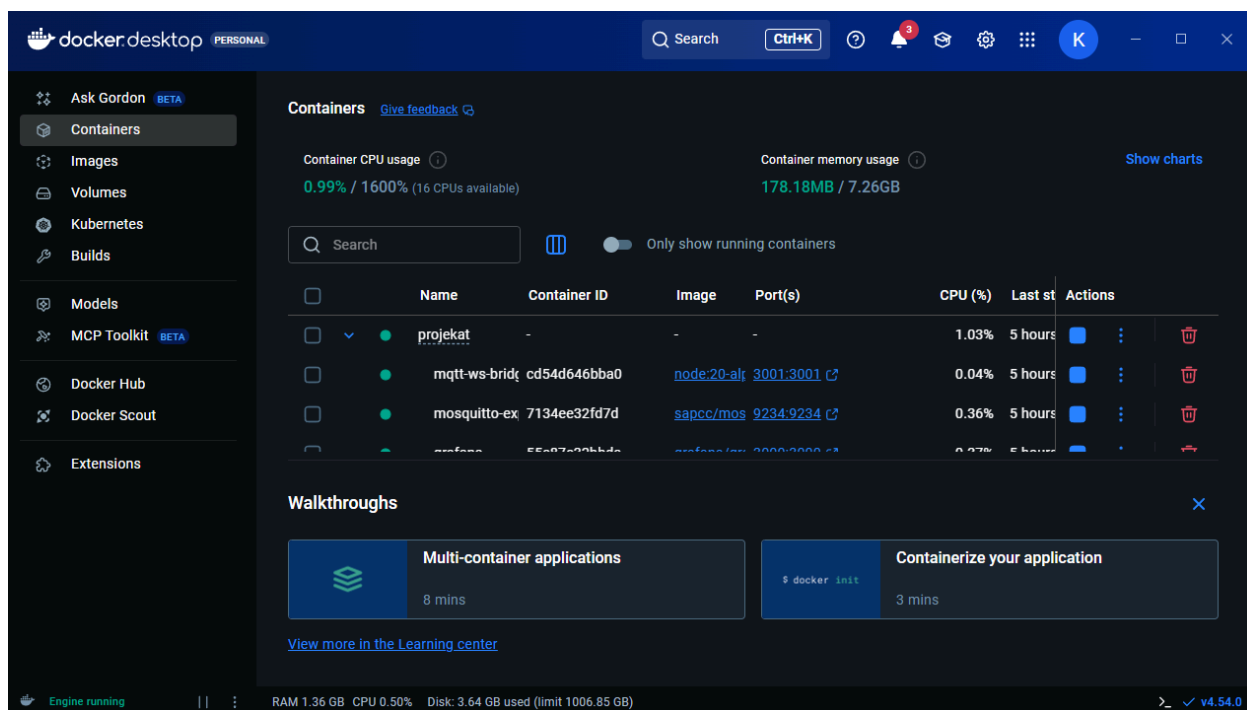
// attachStateInputDemo((code) => indicators.applyDriverState(code));
```


Implementacija je bila dodana v skupni Git repozitorij, kar omogoča lažje sledenje spremembam in usklajevanje dela med člani projektne skupine.



Integracija z Dockerom in “fine tuning” aplikacije

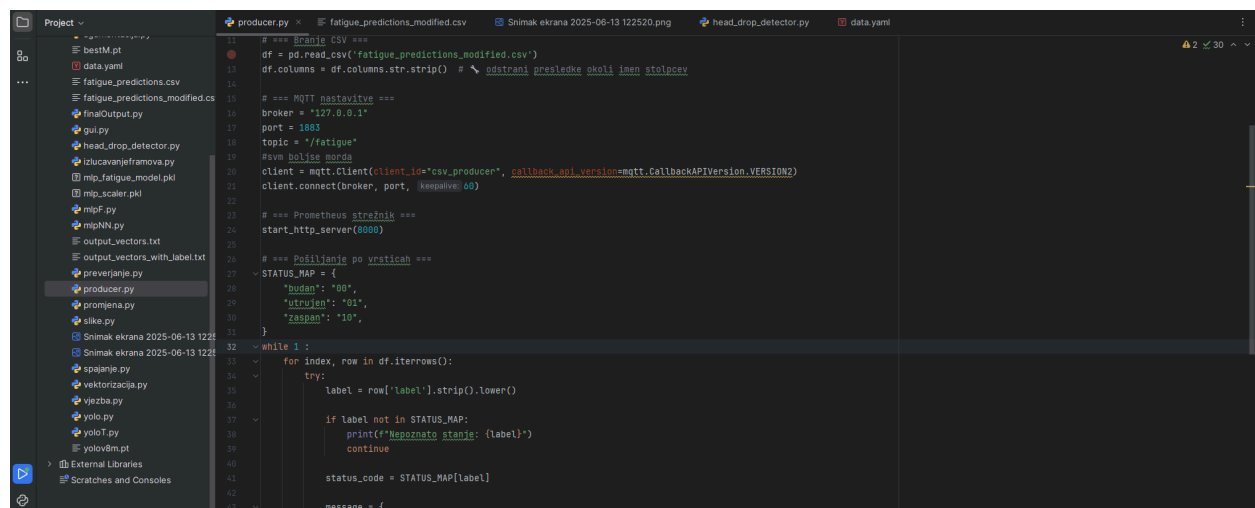
Moja naloga je bila integracija z sistemom iz prejšnjega letnika. Naredil sem MQTT strežnik, prometheus, grafanu, mqtt exporter, mqtt bridge v Docker Containeru.



Tudi sam implementiraj docker.yml file:

```
52 bridge:
53   image: node:20-alpine
54   container_name: mqtt-ws-bridge
55   working_dir: /app
56   volumes:
57     - ./bridge:/app
58   command: sh -c "npm i && node bridge.js"
59   ports:
60     - "3001:3001"
61   environment:
62     - MQTT_URL=mqtt://mosquitto:1883
63     - MQTT_TOPIC=/fatigue
64   depends_on:
65     - mosquitto
66   networks:
67     - mqtt-net
68
69 networks:
70   mqtt-net: # Ustvarimo omrežje z imenom mqtt-net
71     name: mqtt-net
```

In [producer.py](#) ki pošilja podatke na /fatigue topic na mqtt strežnik pa potem gre na mqtt bridge od kod aplikacija bere podatke od 3m modelih.

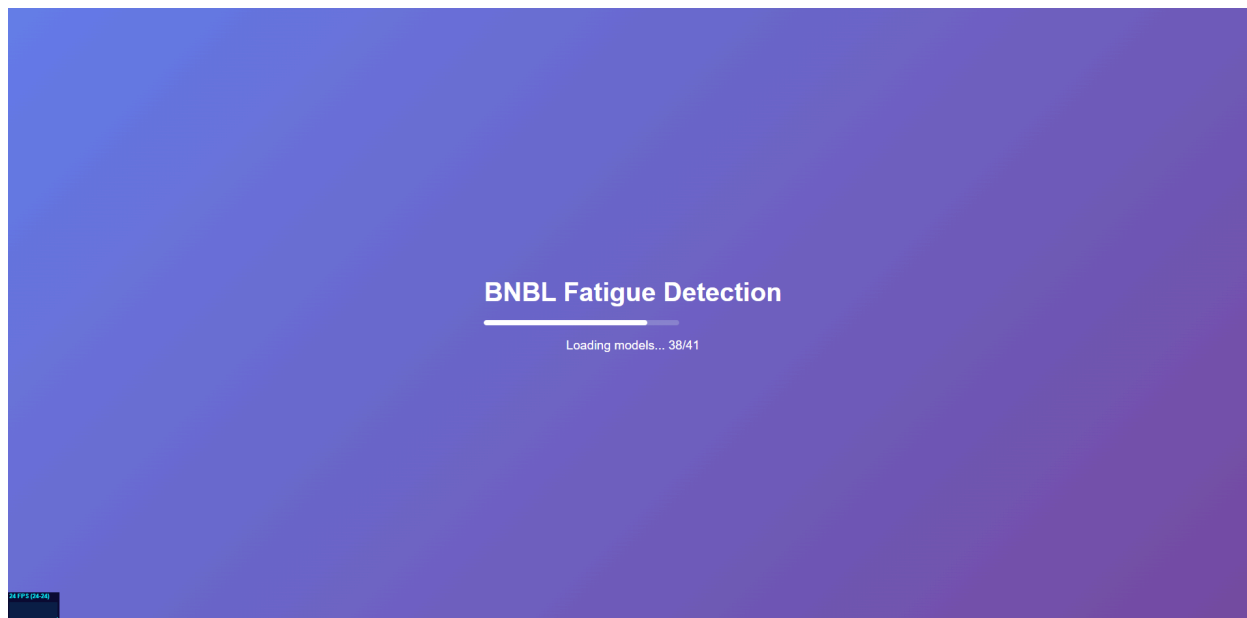


```
11 # === Branje CSV ===
12 df = pd.read_csv('fatigue_predictions_modified.csv')
13 df.columns = df.columns.str.strip() # odstrani presledke okoli imen stolpcev
14
15 # === MQTT publisher ===
16 broker = '127.0.0.1'
17 port = 1883
18 topic = '/fatigue'
19 #vsa bolisa morda
20 client = mqtt.Client(client_id='csv_producer', callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
21 client.connect(broker, port, keepalive=60)
22
23 # === Prometheus strežnik ===
24 start_http_server(8080)
25
26 # === Pošiljanje po vrsticah ===
27 STATUS_MAP = {
28     'budno': '00',
29     'utrujen': '01',
30     'zaspal': '10',
31 }
32
33 while 1:
34     for index, row in df.iterrows():
35         try:
36             label = row['label'].strip().lower()
37
38             if label not in STATUS_MAP:
39                 print(f'Nepoznano stanje: {label}')
40                 continue
41
42             status_code = STATUS_MAP[label]
43
44             message = {
```

Fine tuning aplikacije:

Dodal sem nove funkcionalnosti

- Nalagalni zaslon - Prikaže vizualno privlačen gradient zaslon med nalaganjem aplikacije. Progresni trak dinamično prikazuje odstotek naloženih 3D modelov (npr. "Loading models... 3/5"), kar uporabniku omogoča sledenje napredku. Po končanem nalaganju se zaslon samodejno skrije z glatkim fade-out efektom.



```

// Loading manager for progress tracking
const loadingManager = new THREE.LoadingManager();
const progressBar = document.getElementById('progress-fill');
const loadingText = document.getElementById('loading-text');
const loadingScreen = document.getElementById('loading-screen');

let itemsLoaded = 0;
let itemsTotal = 0;

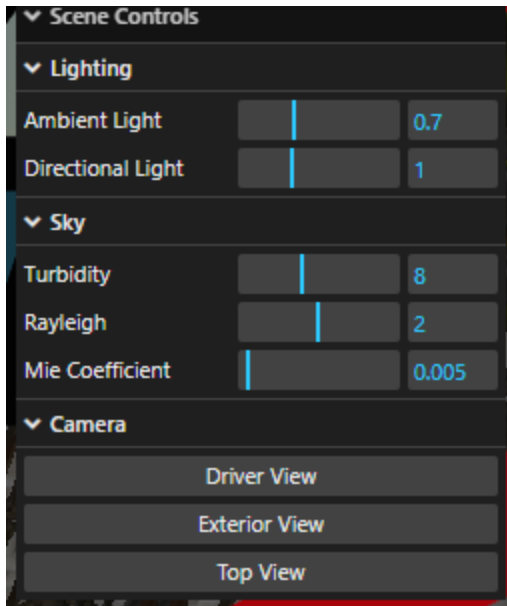
loadingManager.onStart = (url, loaded, total) => {
  itemsTotal = total;
  console.log(`Loading started: ${loaded}/${total}`);
};

loadingManager.onProgress = (url, loaded, total) => {
  itemsLoaded = loaded;
  const progress = (loaded / total) * 100;
  progressBar.style.width = progress + '%';
  loadingText.textContent = `Loading models... ${loaded}/${total}`;
};

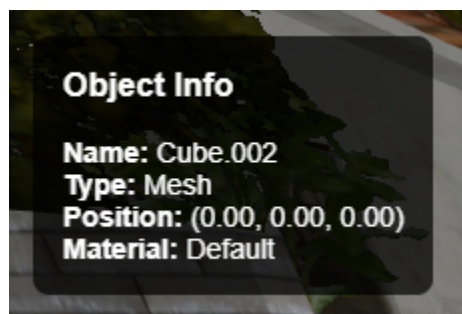
loadingManager.onLoad = () => {
  console.log('All models loaded!');
  loadingText.textContent = 'Complete!';
  setTimeout(() => {
    loadingScreen.classList.add('hidden');
  }, 500);
};

```

- FPS števec - Stats monitor v spodnjem levem kotu prikazuje število sličic na sekundo (FPS), čas renderiranja in porabo pomnilnika. To orodje je ključno za spremljanje zmogljivosti aplikacije in optimizacijo 3D scene.
- GUI kontrolni panel - Interaktivni vmesnik (lil-gui knjižnica) v zgornjem desnem kotu omogoča prilagajanje parametrov v realnem času. Uporabnik lahko spreminja intenziteto ambient in directional svetlobe, prilagaja parametre neba (turbidity, rayleigh, mie coefficient) ter preklaplja med prednastavljenimi pogledi kamere.



- Klik interakcija - Raycasting tehnologija omogoča klikanje na poljuben 3D objekt v sceni. Ob kliku se prikaže informacijski panel z detajli objekta: ime, tip (Mesh/Group), 3D pozicija (x, y, z koordinate) in ime materiala. Panel se samodejno skrije ob kliku na prazen prostor.



- Gladke animacije kamere - Implementirane smooth transitions med različnimi pogledi kamere z easing funkcijo (ease-in-out cubic). Tri predstavljeni pogledi: Driver View (originalni pogled voznika), Exterior View (zunanji pogled na sceno) in Top View (pogled od zgoraj). Animacija traja 1.5 sekunde.
- WebSocket status indikator - Barvni indikator v zgornjem desnem kotu prikazuje stanje povezave z Docker strežnikom (zelena = povezano, rdeča = nepovezano). Sistem samodejno poskuša ponovno vzpostaviti povezavo vsakih 5 sekund ob prekinitvi, kar zagotavlja zanesljivo delovanje real-time komunikacije.
- Izboljšane sence - Aktiviran shadow mapping sistem z visoko resolucijo (2048x2048) in PCF soft shadow algoritmom. Directional svetloba zdaj meče realistične mehke sence, kar izboljša globino in prostorsko zaznavo 3D scene.

```

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
document.body.style.margin = "0";
document.body.appendChild(renderer.domElement);

// Stats monitor (FPS counter)
const stats = new Stats();
stats.dom.style.position = 'fixed';
stats.dom.style.bottom = '0px';
stats.dom.style.top = 'auto';
document.body.appendChild(stats.dom);

const ambientLight = new THREE.AmbientLight(0xffffff, 0.7);
scene.add(ambientLight);

const dir = new THREE.DirectionalLight(0xffffff, 1.0);
dir.position.set(5, 10, 5);
dir.castShadow = true;
dir.shadow.mapSize.width = 2048;
dir.shadow.mapSize.height = 2048;
scene.add(dir);

```

Prikaz sprotnega dela:

Commits on Dec 14, 2025		
Create workspace.xml	kowe18 committed yesterday	76ee27f <>
Commits on Dec 15, 2025		
project setup	milos-avakumovic committed yesterday	4def6b0 <>
adding car + control unit	milos-avakumovic committed yesterday	c344f8c <>
adding sky and mounted camera	milos-avakumovic committed yesterday	4c93d15 <>
added surrounding	milos-avakumovic committed yesterday	d8e24c3 <>
Add WebSocket-based fatigue indicators for control unit	petrovicsladjana committed 6 hours ago	db2537a <>
little changes for app	kowe18 committed 4 hours ago	347378c <>
Implementation of Mqtt for this subject	kowe18 committed 4 hours ago	d41bdbc <>
Commits on Dec 16, 2025		
Added arms and moving birds	milos-avakumovic committed 1 hour ago	7d48d7e <>